

PRACTICAL 29 -

Objective - Operations on Graph

1. **Graph Representation (Adjacency Matrix & List)**

Write a program to represent a graph using both adjacency list and adjacency matrix.

2. **Breadth-First Search (BFS)**

Perform BFS traversal of a graph from a given starting node.

3. **Depth-First Search (DFS)**

Implement DFS traversal of a graph using recursion and stack.

4. **Detect Cycle in an Undirected Graph**

Use DFS or BFS to detect if a cycle exists in an undirected graph.

5. **Detect Cycle in a Directed Graph**

Detect cycle using DFS and back edge technique in a directed graph.

6. **Check if a Graph is Connected**

Given a graph, check if all vertices are reachable from a starting point.

CODE -

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  // Define the structure for adjacency list node
6  typedef struct AdjListNode {
7      int vertex;
8      struct AdjListNode* next;
9  } AdjListNode;
10
11 // Define the structure for adjacency list
12 typedef struct AdjList {
13     AdjListNode* head;
14 } AdjList;
15
16 // Define the structure for graph
17 typedef struct Graph {
18     int numVertices;
19     int** adjMatrix;
20     AdjList* adjLists;
21 } Graph;
22
23 // Create a new graph
```

```
24 Graph* create_graph(int numVertices) {
25     Graph* graph = (Graph*)malloc(sizeof(Graph));
26     graph->numVertices = numVertices;
27
28     // Initialize adjacency matrix
29     graph->adjMatrix = (int**)malloc(numVertices * sizeof(int*));
30     for (int i = 0; i < numVertices; i++) {
31         graph->adjMatrix[i] = (int*)calloc(numVertices, sizeof(int));
32     }
33
34     // Initialize adjacency lists
35     graph->adjLists = (AdjListNode*)malloc(numVertices * sizeof(AdjListNode));
36     for (int i = 0; i < numVertices; i++) {
37         graph->adjLists[i].head = NULL;
38     }
39
40     return graph;
41 }
42
43 // Create a new adjacency list node
44 AdjListNode* create_adj_list_node(int vertex) {
45     AdjListNode* newNode = (AdjListNode*)malloc(sizeof(AdjListNode));
46     newNode->vertex = vertex;
47     newNode->next = NULL;
48     return newNode;
49 }
50
51 // Add an edge to the graph (both adjacency matrix and list)
52 void add_edge(Graph* graph, int src, int dest) {
53     // Add to adjacency matrix
54     graph->adjMatrix[src][dest] = 1;
55     graph->adjMatrix[dest][src] = 1; // For undirected graph
56
57     // Add to adjacency list
58     AdjListNode* newNode = create_adj_list_node(dest);
59     newNode->next = graph->adjLists[src].head;
60     graph->adjLists[src].head = newNode;
61
62     newNode = create_adj_list_node(src);
63     newNode->next = graph->adjLists[dest].head;
64     graph->adjLists[dest].head = newNode;
65 }
66
67 // Display adjacency matrix
68 void display_adj_matrix(Graph* graph) {
69     printf("Adjacency Matrix:\n");
70     for (int i = 0; i < graph->numVertices; i++) {
71         for (int j = 0; j < graph->numVertices; j++) {
72             printf("%d ", graph->adjMatrix[i][j]);
73         }
74         printf("\n");
75     }
```

```
75     }
76 }
77
78 // Display adjacency list
79 void display_adj_list(Graph* graph) {
80     printf("Adjacency List:\n");
81     for (int i = 0; i < graph->numVertices; i++) {
82         printf("%d: ", i);
83         AdjListNode* temp = graph->adjLists[i].head;
84         while (temp) {
85             printf("%d -> ", temp->vertex);
86             temp = temp->next;
87         }
88         printf("NULL\n");
89     }
90 }
91
92 // Breadth-First Search (BFS)
93 void bfs(Graph* graph, int startVertex) {
94     bool visited[graph->numVertices];
95     for (int i = 0; i < graph->numVertices; i++) visited[i] = false;
96
97     int queue[graph->numVertices];
98     int front = 0, rear = 0;
99
100     visited[startVertex] = true;
101     queue[rear++] = startVertex;
102
103     printf("BFS Traversal: ");
104     while (front < rear) {
105         int currentVertex = queue[front++];
106         printf("%d ", currentVertex);
107
108         AdjListNode* temp = graph->adjLists[currentVertex].head;
109         while (temp) {
110             int adjVertex = temp->vertex;
111             if (!visited[adjVertex]) {
112                 visited[adjVertex] = true;
113                 queue[rear++] = adjVertex;
114             }
115             temp = temp->next;
116         }
117     }
118     printf("\n");
119 }
120
121 // Depth-First Search (DFS) Recursive
122 void dfs_recursive_util(Graph* graph, int vertex, bool visited[]) {
123     visited[vertex] = true;
124     printf("%d ", vertex);
125 }
```

```

126     AdjListNode* temp = graph->adjLists[vertex].head;
127     while (temp) {
128         if (!visited[temp->vertex]) {
129             dfs_recursive_util(graph, temp->vertex, visited);
130         }
131         temp = temp->next;
132     }
133 }
134
135 void dfs_recursive(Graph* graph, int startVertex) {
136     bool visited[graph->numVertices];
137     for (int i = 0; i < graph->numVertices; i++) visited[i] = false;
138
139     printf("DFS Traversal (Recursive): ");
140     dfs_recursive_util(graph, startVertex, visited);
141     printf("\n");
142 }
143
144 // Depth-First Search (DFS) Iterative
145 void dfs_iterative(Graph* graph, int startVertex) {
146     bool visited[graph->numVertices];
147     for (int i = 0; i < graph->numVertices; i++) visited[i] = false;
148
149     int stack[graph->numVertices];
150     int top = -1;
151
152     stack[++top] = startVertex;
153
154     printf("DFS Traversal (Iterative): ");
155     while (top != -1) {
156         int currentVertex = stack[top--];
157
158         if (!visited[currentVertex]) {
159             visited[currentVertex] = true;
160             printf("%d ", currentVertex);
161         }
162
163         AdjListNode* temp = graph->adjLists[currentVertex].head;
164         while (temp) {
165             if (!visited[temp->vertex]) {
166                 stack[++top] = temp->vertex;
167             }
168             temp = temp->next;
169         }
170     }
171     printf("\n");
172 }
173
174 // Detect cycle in an undirected graph using DFS
175 bool detect_cycle_undirected_dfs(Graph* graph, int vertex, bool visited
    [], int parent) {

```

```
176     visited[vertex] = true;
177
178     AdjListNode* temp = graph->adjLists[vertex].head;
179     while (temp) {
180         if (!visited[temp->vertex]) {
181             if (detect_cycle_undirected_dfs(graph, temp->vertex,
182                 visited, vertex)) {
183                 return true;
184             }
185         } else if (temp->vertex != parent) {
186             return true;
187         }
188         temp = temp->next;
189     }
190     return false;
191 }
192
193 bool detect_cycle_undirected(Graph* graph) {
194     bool visited[graph->numVertices];
195     for (int i = 0; i < graph->numVertices; i++) visited[i] = false;
196
197     for (int i = 0; i < graph->numVertices; i++) {
198         if (!visited[i]) {
199             if (detect_cycle_undirected_dfs(graph, i, visited, -1)) {
200                 return true;
201             }
202         }
203     }
204     return false;
205 }
206
207 // Detect cycle in a directed graph using DFS
208 bool detect_cycle_directed_dfs(Graph* graph, int vertex, bool visited
209     [], bool recStack[]) {
210     if (!visited[vertex]) {
211         visited[vertex] = true;
212         recStack[vertex] = true;
213
214         AdjListNode* temp = graph->adjLists[vertex].head;
215         while (temp) {
216             if (!visited[temp->vertex] && detect_cycle_directed_dfs(
217                 graph, temp->vertex, visited, recStack)) {
218                 return true;
219             } else if (recStack[temp->vertex]) {
220                 return true;
221             }
222             temp = temp->next;
223         }
224     }
225     recStack[vertex] = false;
226     return false;
227 }
```

```
224 }
225
226 bool detect_cycle_directed(Graph* graph) {
227     bool visited[graph->numVertices];
228     bool recStack[graph->numVertices];
229     for (int i = 0; i < graph->numVertices; i++) {
230         visited[i] = false;
231         recStack[i] = false;
232     }
233
234     for (int i = 0; i < graph->numVertices; i++) {
235         if (detect_cycle_directed_dfs(graph, i, visited, recStack)) {
236             return true;
237         }
238     }
239     return false;
240 }
241
242 // Check if a graph is connected
243 bool is_connected(Graph* graph) {
244     bool visited[graph->numVertices];
245     for (int i = 0; i < graph->numVertices; i++) visited[i] = false;
246
247     // Perform DFS from the first vertex
248     dfs_recursive_util(graph, 0, visited);
249
250     // Check if all vertices are visited
251     for (int i = 0; i < graph->numVertices; i++) {
252         if (!visited[i]) return false;
253     }
254     return true;
255 }
256
257 // Main function to demonstrate graph operations
258 int main() {
259     int numVertices = 5;
260     Graph* graph = create_graph(numVertices);
261
262     add_edge(graph, 0, 1);
263     add_edge(graph, 0, 2);
264     add_edge(graph, 1, 3);
265     add_edge(graph, 1, 4);
266
267     display_adj_matrix(graph);
268     display_adj_list(graph);
269
270     bfs(graph, 0);
271     dfs_recursive(graph, 0);
272     dfs_iterative(graph, 0);
273 }
```

```
274     printf("Cycle in Undirected Graph: %s\n", detect_cycle_undirected(  
        graph) ? "Yes" : "No");  
275     printf("Cycle in Directed Graph: %s\n", detect_cycle_directed(graph  
        ) ? "Yes" : "No");  
276     printf("Graph is Connected: %s\n", is_connected(graph) ? "Yes" : "  
        No");  
277  
278     return 0;  
279 }
```

OUTPUT -

Adjacency Matrix:

0 1 1 0 0

1 0 0 1 1

1 0 0 0 0

0 1 0 0 0

0 1 0 0 0

Adjacency List:

0: 2 -> 1 -> NULL

1: 4 -> 3 -> 0 -> NULL

2: 0 -> NULL

3: 1 -> NULL

4: 1 -> NULL

BFS Traversal: 0 2 1 4 3

DFS Traversal (Recursive): 0 2 1 4 3

DFS Traversal (Iterative): 0 1 3 4 2

Cycle in Undirected Graph: No

Cycle in Directed Graph: Yes

0 2 1 4 3 Graph is Connected: Yes